

Getting Started - Your First RESTful API Query

The following URL shows a sample query for Petrale sole (scientific name = *Eopsetta jordani*), against the trawl survey catch data, from 2010 until 2012:

json output

[https://www.webapps.nwfsc.noaa.gov/trips/api/v1/source/rawl.catch_fact/selection.json?filters=field_identified_taxonomy_dim\\$scientific.name=Eopsetta%20jordani,date_dim\\$year>=2010,date_dim\\$year<=2012&variables=date_YYYYMMDD.field_id_identified_taxonomy_dim\\$scientific.name](https://www.webapps.nwfsc.noaa.gov/trips/api/v1/source/rawl.catch_fact/selection.json?filters=field_identified_taxonomy_dim$scientific.name=Eopsetta%20jordani,date_dim$year>=2010,date_dim$year<=2012&variables=date_YYYYMMDD.field_id_identified_taxonomy_dim$scientific.name)

Click on it and you should get back a stream of json-formatted data in your web browser that shows the date and scientific name that looks like the following:

Sample JSON output

```
[
  {
    "date_yyyymmdd": "20121009",
    "field_identified_taxonomy_dim$scientific_name": "Eopsetta jordani"
  },
  {
    "date_yyyymmdd": "20100822",
    "field_identified_taxonomy_dim$scientific_name": "Eopsetta jordani"
  },
  {
    "date_yyyymmdd": "20100822",
    "field_identified_taxonomy_dim$scientific_name": "Eopsetta jordani"
  },
  {
    "date_yyyymmdd": "20100823",
    "field_identified_taxonomy_dim$scientific_name": "Eopsetta jordani"
  },
  {
    "date_yyyymmdd": "20100823",
    "field_identified_taxonomy_dim$scientific_name": "Eopsetta jordani"
  },
  {
    "date_yyyymmdd": "20100823",
    "field_identified_taxonomy_dim$scientific_name": "Eopsetta jordani"
  },
  {
    "date_yyyymmdd": "20100824",
    "field_identified_taxonomy_dim$scientific_name": "Eopsetta jordani"
  }
]
```

If you'd prefer a csv download of the data, change `selection.json` to `selection.csv` as follows:

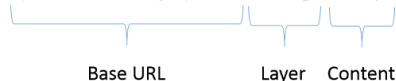
csv output

[https://www.webapps.nwfsc.noaa.gov/trips/api/v1/source/rawl.catch_fact/selection.csv?filters=field identified taxonomy_dim\\$scientific name=Eopsetta%20jordani,date_dim\\$year>=2010,date_dim\\$year<=2012&variables=date_YYYYMMDD,field identified taxonomy_dim\\$scientific name](https://www.webapps.nwfsc.noaa.gov/trips/api/v1/source/rawl.catch_fact/selection.csv?filters=field identified taxonomy_dim$scientific name=Eopsetta%20jordani,date_dim$year>=2010,date_dim$year<=2012&variables=date_YYYYMMDD,field identified taxonomy_dim$scientific name)

Dissecting the URL

Now let's dissect the components of the URL to understand what is happening:

http://nwcdevfram.nwfsc.noaa.gov/api/v1/source/trawl.catch_fact/selection.json?



```
filters=field_identified_taxonomy_dim$scientific_name=Eopsetta%20jordani,date_dim$year>=2010,date_dim$year<=2012&
```



Detailed Reference Documentation

Overview

The FRAM Data Warehouse provides a rich RESTful API that can be used by external software programs to retrieve data. Designed as a data warehouse, it contains two types of tables:

- **Dimension Tables** - These contain the parameters by which users typically want to query the data. For instance, date and time values are dimension tables.
- **Fact Tables** - These contain the measured values of the information that FRAM has collected. For instance, the trawl survey catch data is a fact table.

URL Endpoints

There are three primary URL endpoints in the API. These are RESTful URLs and, by appending parameters to these URLs, enable users to directly query against the data warehouse. The endpoints are:

Source

Critically important for each layer identified in the source URL is the id element. This id element is what is subsequently used to query for layer variables and actual data from the layers. Each entry looks like the following:

Source sample layer entry

```
{
  "id": "warehouse.latitude_dim",
  "project": "warehouse",
  "source": "latitude_dim",
  "contact": "N/A",
  "notes": "N/A",
  "updated": "2016-01-22T17:22:45.199689Z",
  "type": "d",
  "uri": "/warehouse/latitude_dim"
}
```

In this instance, the id is warehouse.latitude_dim. This means that we’re looking at a dimension table (notice the _dim suffix) that contains latitude information. Alternatively, an id can end in _fact or _view that represent fact tables and views respectively. (Our fact views are actually database views built on top of other fact tables, thus the _view suffix in certain cases).

Selection

The selection endpoint allows a user to actually retrieve the data. A basic selection URL for the trawl survey catch layer looks like the following:

Format	Endpoint	Example
json	selection.json	https://www.webapps.nwfsc.noaa.gov/trips/api/v1/source/rawl.catch_fact/selection.json
csv	selection.csv	https://www.webapps.nwfsc.noaa.gov/trips/api/v1/source/rawl.catch_fact/selection.csv

Query Filters

One can add comma-separated query filters to the ending of the selection endpoint to query for a filtered dataset as follows:

[https://www.webapps.nwfsc.noaa.gov/trips/api/v1/source/rawl.catch_fact/selection.json?filters=field_identified_taxonomy_dim\\$scientific_name=Sebastes%20aurora,date_dim\\$year=2014](https://www.webapps.nwfsc.noaa.gov/trips/api/v1/source/rawl.catch_fact/selection.json?filters=field_identified_taxonomy_dim$scientific_name=Sebastes%20aurora,date_dim$year=2014)

Supported Filtering Operations

The following filter operations are supported:

```
(variable) = (value)
(variable) != (value)
(variable) > (value)
(variable) >= (value)
(variable) < (value)
(variable) <= (value)
(variable) in (value)
(variable) !in (value)
```

~" provides Regular Expression matching e.g.
filters=MyFieldName=~321 - matches any occurrence of "321" in the field's text
filters=MyFieldName=~^321\$ - matches any row with exactly value "321" in the field
filters=MyFieldName=~^321|^987\$ - matches any row with exactly value "321" in the field or value "987"

"|" provides OR matching with robust type support, via an urlencoded comma (%2C) separated list of quoted string values enclosed in square brackets e.g. :
filters=MyFieldName=["321"] - matches any record with exactly a value of 321 (converted from a string, into the type of the field).
filters=MyFieldName=["321"%2C"987"] - matches any records with an exact value of either 321 or 987.
"